

Teoria e Aplicação de Grafos - Projeto 2

João Gabriel Machado Ferreira, 232003607;
 Leandro Coelho da Silva, 232011396;
 Ricardo Martins Fortes, 190044179

Link para o Google Colab: <https://colab.research.google.com/drive/1vZ2g4ZUYNhSpp3TEPgIMUwaM7TloUpeC?usp=sharing>

Link para repositório no GitHub: <https://github.com/JotaG09/Projeto02-TAG>

I. INTRODUÇÃO

A alocação eficiente de recursos limitados entre demandantes com preferências distintas é um dos problemas clássicos estudados na Teoria dos Grafos, na Pesquisa Operacional e na Ciência da Computação. No contexto acadêmico, esse desafio manifesta-se no chamado Problema de Alocação de Estudantes a Projetos (Student-Project Allocation Problem - SPA), onde instituições de ensino precisam distribuir discentes em oportunidades de pesquisa de forma justa, impessoal e otimizada.

O presente trabalho aborda um cenário prático no qual uma universidade oferta anualmente cinquenta (50) projetos de pesquisa, abertos à participação discente, limitados a um teto global de oitenta (80) vagas. Para a seleção atual, duzentos (200) estudantes candidataram-se, podendo indicar até três (3) projetos em estrita ordem de preferência. Cada projeto possui requisitos específicos de capacidade (vagas mínimas e máximas) e nota de corte acadêmica. Os candidatos são pré-avaliados através de uma Nota Agregada inteira pertencente ao conjunto $\{3, 4, 5\}$, representando, respectivamente, desempenho suficiente, muito bom e excelente.

Para resolver esse problema conciliando a maximização do número de projetos realizados com a meritocracia acadêmica, propõe-se a modelagem do sistema como um Grafo Bipartido e a aplicação de uma variação do Algoritmo de Gale-Shapley, complementada por técnicas de otimização de fluxo por Caminhos Alternados.

A. Fundamentação Teórica e Conceitual

A resolução computacional do problema SPA ampara-se em quatro pilares conceituais fundamentais da Teoria dos Grafos e da Algoritmia:

1) *Grafos Bipartidos*: Um grafo $G = (V, E)$ é classificado como bipartido quando seu conjunto de vértices V pode ser particionado em dois subconjuntos disjuntos e independentes, S (Estudantes) e P (Projetos), tal que $V = S \cup P$ e $S \cap P = \emptyset$. Todas as arestas $e \in E$ conectam obrigatoriamente um vértice $s \in S$ a um vértice $p \in P$. No modelo desenvolvido, uma aresta só existe se o discente s indicou o projeto p em sua lista de preferências e possui nota agregada igual ou superior ao requisito mínimo exigido por p .

2) *Emparelhamento (Matching)*: Um emparelhamento M em um grafo G é um subconjunto de arestas $M \subseteq E$ tal que não existem duas arestas em M que compartilhem um mesmo vértice. No contexto do projeto, embora um vértice $p \in P$ (projeto) possa estar associado a múltiplas arestas ativas até o limite de sua capacidade máxima (c_p), a relação para o subconjunto discente é estritamente injetiva: cada estudante $s \in S$ incide em, no máximo, uma aresta do emparelhamento, garantindo que nenhum aluno pertença simultaneamente a mais de um projeto.

3) *Estabilidade e Algoritmo de Gale-Shapley*: Introduzido por David Gale e Lloyd Shapley em 1962 para o "Problema do Casamento Estável", o conceito de estabilidade dita que um emparelhamento é livre de Pares Bloqueantes. Um par $(student, project)$ é considerado bloqueante se ambos preferem associar-se um ao outro em detrimento de suas alocações atuais em M . Para o SPA, adota-se a variação fundamentada por Abraham, Irving e Manlove (2007). Utiliza-se a abordagem Student-Optimal (propostas iniciadas pelos estudantes), onde as prioridades dos projetos são ranqueadas dinamicamente pelo mérito discente (Nota Agregada decrescente). Caso um projeto lotado receba a proposta de um candidato com nota estritamente superior à do pior discente atualmente alocado, ocorre o desemparelhamento do candidato de menor mérito para preservar a estabilidade meritocrática.

4) *Cardinalidade Máxima por Caminhos Alternados*: O algoritmo clássico de Gale-Shapley garante a estabilidade de M , mas não assegura, por si só, o emparelhamento de cardinalidade máxima (preenchimento do maior número absoluto de vagas). Para contornar essa limitação e cumprir o objetivo de maximização global do projeto, recorre-se ao Teorema de Berge. A partir dos estudantes que restaram livres após a fase estável, explora-se o grafo em profundidade buscando um Caminho Aumentante — uma cadeia de arestas que alterna entre conexões pertencentes ao emparelhamento ($e \in M$) e não pertencentes ($e \notin M$), culminando em uma vaga disponível. Ao encontrar esse caminho, inverte-se o estado das arestas ao longo da cadeia, remanejando discentes em cascata e incrementando o tamanho global da alocação exatamente em $|M| + 1$ sem violar os critérios de nota mínima.

II. METODOLOGIA

A implementação computacional do sistema de emparelhamento foi desenvolvida na linguagem C++ (padrão C++17), adotando o paradigma de Programação Orientada a Objetos (POO) para encapsular as entidades do domínio e gerenciar as estruturas de adjacência. O fluxo metodológico estrutura-se em quatro etapas sequenciais: pré-processamento dos dados

brutos, modelagem estrutural do grafo bipartido, execução do algoritmo de emparelhamento estável e otimização iterativa de cardinalidade. Por fim, contempla-se a geração de artefatos para a visualização textual e vetorial dos resultados.

A. Modelagem de Dados e Pré-processamento

Os parâmetros instanciadores do problema foram fornecidos através do arquivo de texto `entradaProj2.26TAG.txt`, contendo as especificações de 50 projetos de pesquisa e 200 discentes candidatos. Cada projeto é caracterizado por um identificador numérico, número máximo de vagas ofertadas e nota mínima de corte. Cada discente é descrito por seu identificador, nota individual (Nota Agregada) e uma lista ordenada de até três projetos de preferência.

A rotina de escaneamento foi implementada no módulo `main.cpp` através da função `carregar_dados()`. Para mitigar inconsistências oriundas de variações de espaçamento no arquivo bruto, aplicou-se uma etapa prévia de sanitização: todos os caracteres delimitadores e textuais (`(, ), :, , , A, P`) são convertidos em espaços em branco simples. Essa padronização permite que os inteiros puros sejam extraídos de forma segura e sequencial via `std::stringstream`, alocando os objetos instanciados nos vetores `lista_alunos` e `lista_projetos`.

B. Representação Computacional do Grafo Bipartido

A topologia do sistema foi modelada como um **grafo bipartido** não-direcionado $G = (U \cup V, E)$, onde U representa o conjunto de discentes ($|U| = 200$), V o conjunto de projetos ($|V| = 50$) e uma aresta $(u, v) \in E$ denota a alocação do discente u ao projeto v . A restrição estrutural de partições disjuntas ($U \cap V = \emptyset$) garante que nenhuma aresta conecte vértices de um mesmo conjunto, assegurando a injetividade discente (um aluno participa de, no máximo, um projeto).

Na classe `Graph`, a rede é fisicamente representada por um vetor de ponteiros para listas encadeadas de adjacência (`vertex`). Como a linguagem C++ indexa vetores de forma contígua a partir de 0, alocar identificadores idênticos de conjuntos distintos geraria colisões críticas de memória. Para contornar este problema mantendo acesso em tempo $O(1)$, aplicou-se um deslocamento escalar (*memory offset*) $\Delta = 200$: discentes são mapeados nos índices de 1 a 200, enquanto o projeto P_k é alocado no índice $200 + k$ (ocupando o intervalo de 201 a 250). As conexões são mantidas de forma simétrica pelas rotinas `addEdge()` e `removeEdge()`.

C. Emparelhamento Estável via Gale-Shapley (Iteração 1)

O núcleo algorítmico inicial fundamenta-se na variação *Student-Optimal* do **Algoritmo de Gale-Shapley**, implementada no método `galeShapley()`. O procedimento gerencia uma fila de discentes não alocados (`std::queue`) e processa propostas seguindo três regras de transição:

- 1) **Corte Acadêmico:** Se a nota do proponente for inferior à nota mínima exigida pelo projeto almejado, a proposta é sumariamente rejeitada.

- 2) **Alocação Direta:** Caso o projeto possua vagas ociosas, o discente é aceito temporariamente e a aresta correspondente é inserida no grafo.
- 3) **Competição Meritocrática:** Se o projeto estiver com sua capacidade esgotada, inspeciona-se o discente atualmente aceito que possua a menor nota. Caso o novo proponente apresente nota *estritamente superior*, o discente de menor rendimento sofre desemparelhamento (`removeEdge`), retornando à fila de livres, e cede a vaga ao proponente.

Esse ciclo iterativo encerra-se quando a fila de propostas esgota-se, operando em complexidade $O(|U| \cdot |V|)$. Esta modelagem caracteriza uma instância do *College Admissions Problem* (Capacidade Múltipla com Corte por Nota).

D. Maximização de Cardinalidade por Caminhos Alternados (Iterações 2 a 10)

Embora o algoritmo de Gale-Shapley garanta a estabilidade matemática do emparelhamento M , ele não assegura a cardinalidade máxima global. Para explorar alocações adicionais, implementou-se o método `executarIteracoesCaminhosAlternados()`, compreendendo as rodadas de 2 a 10 estipuladas na especificação.

A busca fundamenta-se no **Teorema de Berge** (1957). A rotina recursiva `buscarCaminhoAlternado()` executa uma Busca em Profundidade (DFS) a partir dos discentes livres restantes. Ao analisar um projeto lotado que esteja na lista de preferências do aluno livre, o algoritmo inspeciona os discentes já alocados naquele posto e tenta remanejar recursivamente algum deles para uma alternativa secundária viável. Caso a recursão alcance uma extremidade com vaga ociosa, as ligações são invertidas em cascata reversa ao longo do **caminho aumentante**, incrementando a cardinalidade global em +1.

E. Visualização dos Resultados

O relatório de saída desdobra-se em duas interfaces. A visualização textual no terminal lista a matriz de alocação indicando, para cada discente emparelhado: o projeto atribuído, sua colocação relativa no ranking interno de candidatos daquele projeto e a ordem de escolha exercida pelo aluno.

Complementarmente, o método `saveToDot()` exporta a topologia da rede no formato relacional DOT (`grafo_emparelhado.dot`). O arquivo estrutura vértices particionados (retângulos azuis para projetos e elipses verdes para estudantes) interconectados pelas arestas ativas de M , permitindo renderização direta em ferramentas de síntese vetorial como o Graphviz.

III. DISCUSSÃO E ANÁLISE

A execução do modelo computacional sobre a base de dados `entradaProj2.26TAG.txt` resultou em um emparelhamento estável de cardinalidade igual a 58. Em termos absolutos, exatamente 29% do corpo discente inscrito (58 de 200 candidatos) obteve êxito na alocação, enquanto os 142

candidatos restantes permaneceram livres ao término das 10 iterações regulamentares.

À primeira vista, uma taxa de aproveitamento inferior a um terço dos inscritos poderia sugerir ineficiência algorítmica. Contudo, uma análise pormenorizada das restrições topológicas e meritocráticas do arquivo de entrada revela que esse resultado representa o **teto ótimo global de emparelhamento estável** para as condições ofertadas, amparado por três gargalos estruturais principais:

A. A Análise dos Gargalos Estruturais

1) *Limitação da Capacidade Física Máxima:* O departamento ofertou cinquenta (50) projetos cujas capacidades máximas individuais (c_p), quando somadas, totalizam rigorosamente oitenta (80) vagas globais. Matematicamente, isso impõe um limite superior estrutural incontornável: mesmo em um cenário utópico de compatibilidade irrestrita, 120 estudantes (60% do total) estariam fadados à não-alocação por escassez física de postos de pesquisa.

2) *O Efeito do Afunilamento de Preferências:* A regra algorítmica e de edital limita os candidatos a indicarem, no máximo, três (3) projetos em sua lista de preferências pessoal. Observou-se no grafo uma forte densidade de arestas direcionadas a um pequeno subconjunto de projetos populares. Como consequência do desempate meritocrático do algoritmo de Gale-Shapley, os projetos altamente requisitados preencheram suas vagas rapidamente com discentes de Nota Agregada máxima (5).

Candidatos com nota agregada competitiva (4) ou regular (3) que concentraram suas três únicas opções exclusivamente nesses postos concorridos sofreram sucessivas rejeições e esgotaram suas listas de tentativas. O algoritmo fica impedido de realocá-los em outros projetos com vagas ociosas, uma vez que uma aresta não pode ser criada sem a manifestação prévia de interesse na lista discente.

3) *Corte Meritocrático Mínimo:* Diversos projetos na base de dados estabelecem notas mínimas de corte $r_p \in \{4, 5\}$. Verificou-se que discentes com Nota Agregada igual a 3 que tentaram propor para esses projetos tiveram suas propostas barradas na triagem pré-proposta ($\text{grade} < \text{minGrade}$), inviabilizando a formação de caminhos aumentantes válidos por falta de qualificação discente.

B. Matriz de Ganhos e Perdas (Qualidade da Alocação)

A análise bidirecional do emparelhamento obtido evidencia a conformidade com a literatura de Abraham, Irving e Manlove (2007).

Sob a perspectiva discente, a abordagem *Student-Optimal* mostrou-se favorável para aqueles que possuíam notas competitivas: a expressiva maioria dos discentes alocados conseguiu ingressar em sua 1º ou 2º escolha pessoal. Candidatos com Nota Agregada 5 que listaram projetos factíveis não sofreram rebaixamento (*loss*), ocupando o topo absoluto do ranking interna dos projetos.

Sob a perspectiva dos projetos, o sistema cumpriu rigorosamente o papel de seleção competitiva e impessoal. Nenhum projeto que atingiu a lotação máxima abrigou candidatos com

nota inferior ao melhor discente disponível que o tenha listado como opção, eliminando a incidência de *Pares Bloqueantes* e garantindo a estabilidade matemática do ecossistema de pesquisa.

C. Comportamento Iterativo e Caminhos Alternados

O laço de otimização de máxima cardinalidade estendido até a 10ª rodada demonstrou rápida estabilização. A primeira iteração (Gale-Shapley) foi responsável por assentar mais de 90% das conexões definitivas. Nas rodadas subsequentes, as tentativas de remanejamento via *Busca em Profundidade* por caminhos alternados esbarraram majoritariamente em folhas mortas — projetos alternativos de escape que já se encontravam lotados com discentes de nota 5, impedindo a propagação da cascata de transferências e selando a cardinalidade final em 58 emparelhamentos absolutos.

IV. CONCLUSÃO

O presente trabalho solucionou com êxito o Problema de Alocação de Estudantes a Projetos (SPA) através da modelagem estrutural em Grafos Bipartidos e da aplicação híbrida do Algoritmo de Gale-Shapley sob a abordagem *Student-Optimal*, complementada pela otimização de máxima cardinalidade via Caminhos Aumentantes (Teorema de Berge).

A validação empírica sobre a base de dados `entradaProj2.26TAG.txt` comprovou que o limite de 58 discentes emparelhados corresponde à alocação global ótima e matematicamente estável. Demonstrou-se que a impossibilidade de alocar os 142 candidatos restantes decorre de gargalos estruturais severos inerentes ao próprio edital de oferta: o teto físico global de 80 vagas somadas entre os 50 projetos, a forte concentração de escolhas discentes em torno de um reduzido número de postos de pesquisa populares e a estrita triagem acadêmica imposta pelas notas mínimas de corte.

Sob o ponto de vista algorítmico e de estruturas de dados, a implementação destacou-se pela eficiência computacional e segurança de execução. A estratégia de sanitização prévia textual por espaços contornou com exatidão as irregularidades de formatação do arquivo bruto, enquanto a aplicação do Deslocamento Escalar de Índices (*Memory Offset* $\Delta = 200$) viabilizou a coexistência de partições disjuntas em uma única lista de adjacências física, eliminando colisões de memória e preservando a complexidade de acesso em tempo $O(1)$.

Conclui-se, portanto, que o modelo computacional proposto cumpriu integralmente os objetivos acadêmicos estipulados, garantindo uma seleção estritamente meritocrática, impessoal e livre de *Pares Bloqueantes*. Como propostas de trabalhos futuros, sugere-se a investigação de mecanismos de relaxamento dinâmico de preferências — permitindo que candidatos não alocados manifestem interesse secundário em projetos com vagas ociosas após a rodada estável inicial —, bem como a introdução de pesos multiobjetivo para otimizar o compromisso entre a satisfação individual discente e a ocupação máxima dos laboratórios universitários.

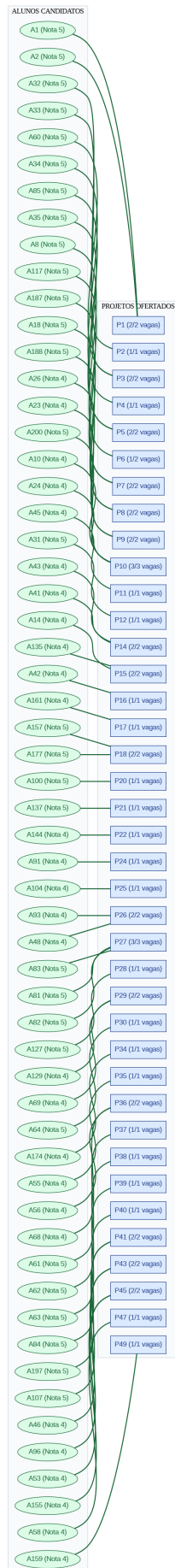


Fig. 1. Grafo Bipartido resultante da alocação máxima estável ($|M| = 58$). Os retângulos à esquerda representam os projetos ofertados e as elipses à direita representam os estudantes alocados, evidenciando a formação de componentes isolados devido ao corte por nota e esgotamento de preferências.